Drew University - CSCI 260 - Fall 2024
HW3 - von Neumann Model & LC-3

HW Info.: Due on 10/11/2024 (F) by 11:59pm, 32 points
Submission: Submit as a single PDF file on Moodle.
Note: Please follow Drew University's Standard of Academic Integrity and refer to the course syllabus for late submission policy.

1. (6 points) Answer questions for the following two 32-bit wide instructions.

   (a) Suppose the instruction has the format below, where both *SR* and *DR* represent register values and *IMM* is the immediate value.

OPCODE	*SR*	*DR*	*IMM*

   If there are in total 60 *OPCODE*s and 32 registers, what is the range of values that can be represented by the *IMM*? Assume *IMM* is in 2's complement.

   Ans: We need 6 bits for the *OPCODE*s, as $2^6 = 64$, and 5 bits for *SR* and *DR*, each, as $2^5 = 32$. That's, in total, 16 bits used. Hence, the *IMM* is 16-bit wide, in which 1 bit is for the sign and 15 bits are for the magnitude. That's $-2^{15} \leq IMM \leq 2^{15} - 1$.

   (b) Suppose the instruction has the format below, where *DR*, *SR1*, and *SR2* all represent register values and *UNUSED* contains bits not used in the instruction.

OPCODE	*DR*	*SR1*	*SR2*	*UNUSED*

   If there are 225 *OPCODE*s and 120 registers, what is the maximum width of *UNUSED*?

   Ans: We need 8 bits for the *OPCODE*s, as $2^8 = 256$, and 7 bits for *DR*, *SR1*, and *SR2*, each, as $2^7 = 128$. Hence, *UNUSED* is 3-bit wide.

2. (12 points) The following table represents a small chunk of a 4-bit addressable memory. Refer to this table for the following questions.

Address	Data
0000	0001 1110 0100 0011
0001	1111 0000 0010 0101
0010	0110 1111 0000 0001
0011	0000 0000 0000 0000
0100	0000 0000 0110 0101
0101	0000 0000 0000 0110
0110	1111 1110 1101 0011
0111	0000 0110 1101 1001

   (a) What hexadecimal values do address locations 3 and 6 contain?

   Ans: Mem[0011] = x0000 and Mem[0110] = xFED3

   (b) Interpret location 4 as ASCII and evaluate its value.

   Ans: Mem[0100] = x0065 = NULL "e" = "e"

(c) The content of memory locations can also be interpreted as instructions. Interpret data stored within location 0 as an instruction, what instruction does it represent?

Ans: Mem[0000] = $(0001\ 111\ 001\ 0\ 00\ 011)_2$ = ADD R7, R1, R3 (or R7 ← R1 + R3)

(d) The content of memory locations can also be interpreted as memory addresses. Interpret data stored within location 5 as an address. Then which location does it refer to? What hexadecimal value does that referred location contain?

Ans: Mem[0101] = $(0000\ 0000\ 0000\ 0110)_2 = (6)_{10}$; Mem[Mem[0101]] = Mem[0110] = xFED3

3. (10 points) Answer the following questions with LC-3 instructions in hexadecimal. **Show all your work on the converting process.**

(a) How could one use a single instruction to move the value in R2 into R3?

Ans: ADD R3, R2, #0 = $(0001\ 011\ 010\ 1\ 00000)_2$ = x16A0; or
AND R3, R2, #-1 = $(0101\ 0110\ 1011\ 1111)_2$ = x56BF

(b) How could one use a single instruction to clear the contents of R5?

Ans: AND R5, R5, #0 = $(0101\ 101\ 101\ 1\ 00000)_2$ = x5B60

(c) There's no subtraction instruction, so how could one perform the **R1 ← R4 − R6** operation?

Ans: We need to use 2's complement for the content of R6, and add that to R4, and the sum is then stored to R1. Hence,

- NOT R6, R6 = $(1001\ 110\ 110\ 111111)_2$ = x9DBF
- ADD R6, R6, #1 = $(0001\ 110\ 110\ 1\ 00001)_2$ = x1DA1
- ADD R1, R4, R6 = $(0001\ 001\ 100\ 0\ 00\ 110)_2$ = x1306

(d) Using only 1 instruction and without changing the contents of any register, how might one set the condition codes based on the value that resides in R1? In fact, there is more than one way of doing so, give two of them.

Ans: Either ADD R1, R1, #0 (x1260), or AND R1, R1, R1 (x5241), or AND R1, R1, #-1 (x527F).

(e) Is there a sequence of instructions that will cause the condition codes at the end of the sequence to be $N = 1,\ Z = 1$, and $P = 0$? Explain in one sentence.

Ans: There's no such a sequence, as the result of that sequence would be both a negative number and zero at the same time.

4. (4 points) There's no logical OR instruction in LC-3. However, by utilizing De Morgan's law, we can obtain it. Recall, NOT(NOT(A) AND NOT(B)) = A OR B.

Suppose we want to perform the operation **R1 ← R2 OR R3**. Write out the 4 instructions in hexadecimal for implementing logical OR. **Show all your work on the converting process.**

Ans:

- NOT R2, R2 = $(1001\ 010\ 010\ 111111)_2$ = x94BF
- NOT R3, R3 = $(1001\ 011\ 011\ 111111)_2$ = x96FF
- AND R1, R2, R3 = $(0101\ 001\ 010\ 0\ 00\ 011)_2$ = x5283
- NOT R1, R1 = $(1001\ 001\ 001\ 111111)_2$ = x927F